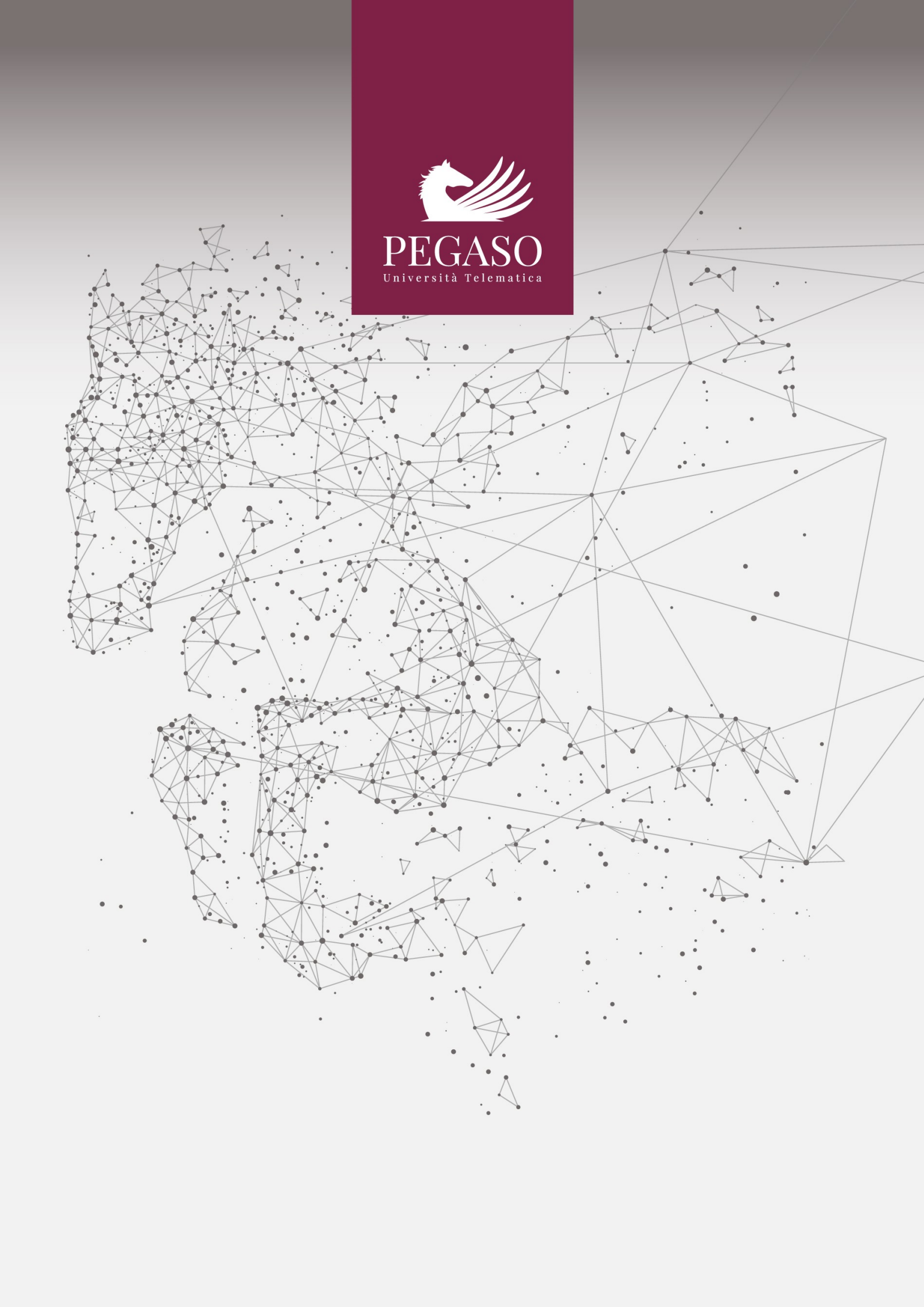

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. SINTASSI DELL'ASSEMBLY MIPS 3
2. ADDIZIONE: ISTRUZIONE ADD 5
3. ADDIZIONE: ISTRUZIONE ADDI 10
4. SOTTRAZIONE: ISTRUZIONE SUB 13
5. ESERCIZI 15
BIBLIOGRAFIA 17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Sintassi dell'assembly MIPS

Le istruzioni di un programma spesso sono dette *comandi* perché, con una visione umanizzata, si potrebbe dire che la macchina le esegue "ciecamente". In altre parole, per la macchina ad ogni istruzione corrisponde una serie di azioni svolte dall'Hardware che non hanno un "significato".

Per evitare ambiguità, le istruzioni sono scritte seguendo le regole precise di un *Linguaggio Formale*. Tali regole costituiscono la **Sintassi** del Linguaggio, ed in base ad esse la macchina determina univocamente le azioni che dovranno esse svolte dall'Hardware.

Quindi, dal punto di vista della macchina, l'unico aspetto dell'istruzione che viene preso in considerazione ai fini dell'esecuzione è la sua Sintassi, e la macchina eseguirà solo istruzioni scritte in forma sintatticamente corretta.

Per il programmatore, invece, le regole sintattiche delle istruzioni sono associate ad un preciso significato, che costituisce la **Semantica** del Linguaggio, relativo alla interpretazione degli effetti che la loro esecuzione produce in termini di elaborazione dell'informazione.

Per ogni istruzione dell'Assembly MIPS che verrà presentata si descriverà, quindi, sia la Sintassi che la Semantica.

STRUTTURA DI BASE DI UNA ISTRUZIONE ASSEMBLY:

- Uso dei simboli dell'alfabeto inglese e della notazione decimale.
- Ogni istruzione è scritta su una sola riga.

  ESEMPIO: `addi $t3, $t1, –570`
- Ogni istruzione contiene:
  - una sequenza alfanumerica iniziale, detta **Codice Operativo**, che dal punto di vista semantico rappresenta il tipo di azione da eseguire e come reperire gli operandi,

    ESEMPIO: la sequenza "addi" è un Codice Operativo.
  - altri simboli che consentono di stabilire dove si trovano gli operandi e dove memorizzare il risultato.

    ESEMPIO: i simboli `$t3, $t1, –570`.

È possibile associare un **commento** ad un'istruzione ponendolo sulla stessa riga dell'istruzione e facendolo iniziare con il simbolo "#".

Non c'è un simbolo di fine commento, quindi, se il commento è lungo ed occupa più righe, ogni riga deve iniziare con il simbolo "#".

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO:

```
add   $t0,  $s1,  $s0        # questo è un commento
                             # scritto su due righe.
```

Il programma Assembler, che effettua la traduzione in Linguaggio Macchina, leggendo il simbolo "#" tralascerà l'analisi di tutto quello che segue il simbolo e passerà alla riga successiva.

Se questa è un commento verrà ancora tralasciata passando alla riga successiva, fino a quando viene trovato un Codice Operativo da cui inizia la traduzione dell'istruzione.

I commenti costituiscono un aiuto per la comprensione di un programma Assembly che, a causa del formalismo sintattico, tende ad essere rapidamente poco comprensibile con l'aumentare del numero delle istruzioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Addizione: istruzione add

Qualsiasi computer consente di eseguire operazioni aritmetiche. Per una stessa operazione possono essere disponibili più istruzioni Assembly che distinguono diverse modalità per individuare gli operandi. Questo è il caso dell'addizione nell'Assembly MIPS per la quale sono disponibili due diverse istruzioni individuate dai Codici Operativi "add" ed "addi".

ISTRUZIONE add

Effettua l'addizione di due operandi interi con segno il cui valore è memorizzato in due registri e scrive il risultato in un registro.

Per individuare i registri che contengono gli operanti ed il risultato, all'interno dell'istruzione viene fornito il rispettivo indirizzo in precise posizioni.

I registri utilizzati per contenere dati sono denotati con le lettere "t" ed "s" seguite da un numero. La seguente tabella indica la corrispondenza tra gli indirizzi numerici e la notazione simbolica:

simbolo	indirizzo	simbolo	indirizzo	simbolo	indirizzo
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

I valori degli operandi devono essere già presenti nei registri prima di eseguire l'istruzione, e deve essere specificato quale è il registro in cui verrà memorizzato il risultato detto *registro destinazione*.

L'addizione è una operazione disponibile come istruzione in tutti i Linguaggi ad Alto Livello. Nel **Linguaggio C** è codificata da una istruzione del tipo:

```
a = b + c  ;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per tradurre una istruzione in Assembly MIPS, occorre prima assegnare alle variabili "b" e "c", che rappresentano gli operandi, i registri che dovranno contenere i valori delle variabili prima dell'esecuzione dell'istruzione. Alla variabile "a" deve essere assegnato il registro destinazione in cui verrà scritto il risultato ottenuto dall'esecuzione dell'istruzione.

Assegnando alle variabili "a", "b", "c" i registri con i seguenti INDIRIZZI:

a	b	c	*variabile*
↓	↓	↓	
$t0	$s1	$s2	*indirizzo simbolico*
↓	↓	↓	
8	17	18	*indirizzo numerico*

l'istruzione in Linguaggio C viene tradotta dalla seguente **istruzione Assembly**:

```
add  $t0, $s1, $s2
```

Vincoli sintattici:

- Codice Operativo **add**,
- l'**indirizzo del registro DESTINAZIONE** che conterrà il risultato è il primo, ed è separato da una virgola dai successivi indirizzi,
- gli **indirizzi dei due REGISTRI DEGLI OPERANDI** seguono l'indirizzo destinazione e sono separati da una virgola.

L'esecuzione dell'istruzione ha effetto sul contenuto del registro destinazione, ma lascia immutato il contenuto dei registri degli operandi.

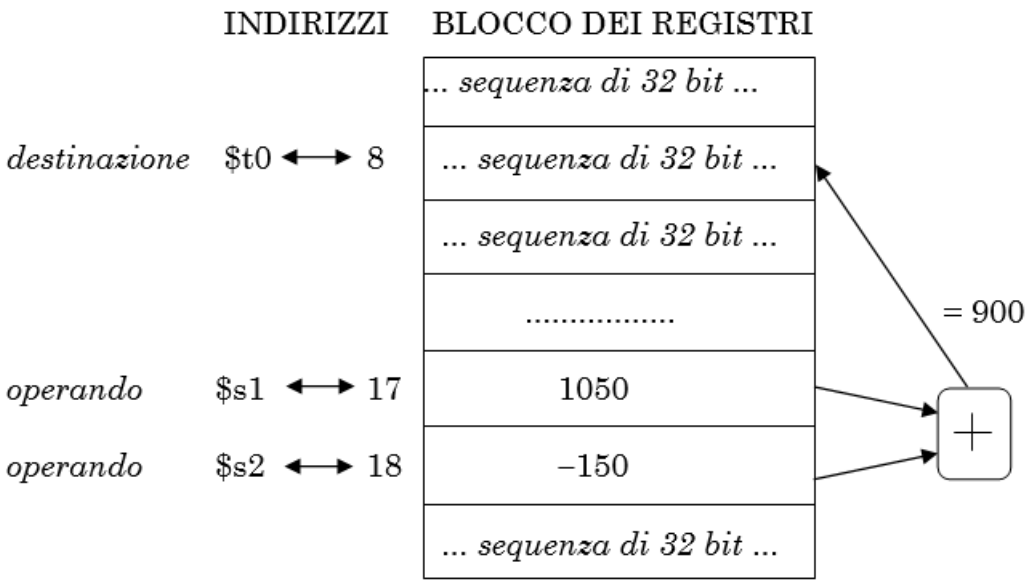
Ad esempio, supponendo che i valori degli operandi della precedente istruzione siano:

$$b = 1050 \quad c = -150 \quad \Longrightarrow \quad a = b + c = 1050 + (-150) = 900$$

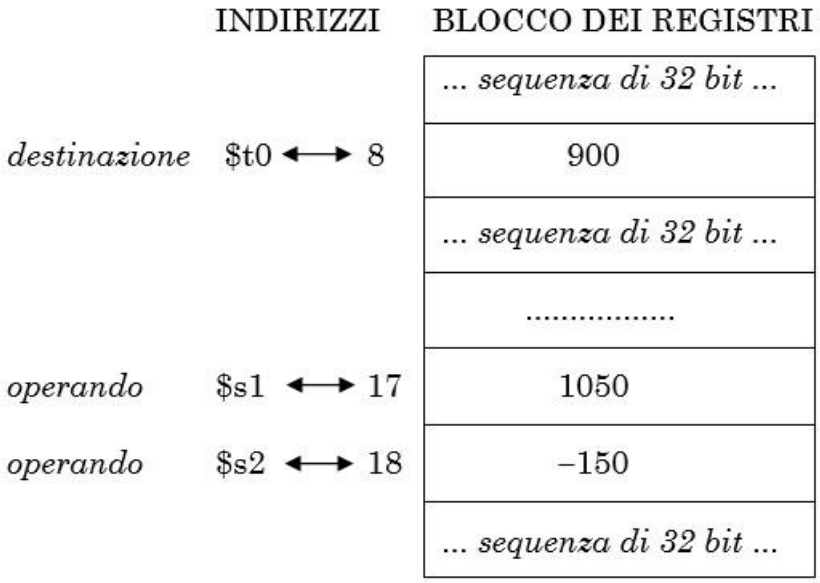
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

il contenuto nei registri **PRIMA dell'esecuzione dell'istruzione** è il seguente:



il contenuto nei registri **DOPO l'esecuzione dell'istruzione** è il seguente:



Nelle figure i contenuti sono indicati con simboli alfanumerici e con l'usuale notazione decimale. La notazione effettivamente adottata all'interno del computer per la rappresentazione degli interi con segno è la *Notazione in complemento a 2*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Così come accade nel Linguaggio C, anche nell'Assembly il registro destinazione può essere uguale a quello di uno degli operandi (o di entrambi) come mostrato nei due seguenti esempi.

ESEMPIO_A: consideriamo l'istruzione C:

```
a = a + c ;
```

con le assegnazioni tra variabile e indirizzo date da:

$$a \rightarrow \$t7 \qquad c \rightarrow \$s5$$

la traduzione Assembly è data da:

```
add  $t7,  $t7,  $s5        #  calcola  a = a + c
```

L'esecuzione dell'istruzione determina la cancellazione di uno degli operandi per effetto della scrittura del risultato nel registro che lo contiene.

Ad esempio, supponendo che i valori degli operandi siano:

$$a = -430 \quad c = -70 \quad \Longrightarrow \quad a = a + c = -430 + (-70) = -500$$

si ha il seguente contenuto nei registri prima e dopo l'esecuzione dell'istruzione:

INDIRIZZI	BLOCCO DEI REGISTRI **PRIMA**	BLOCCO DEI REGISTRI **DOPO**
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*
$t7 ⟷ 15	–430	–500
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*
	.....	.....
$s5 ⟷ 21	–70	–70
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*

ESEMPIO_B: consideriamo l'istruzione C

```
b = b*2  ;
```

in modo equivalente si può calcolare: b = b*2 = b+b.

Quindi con l'assegnazione tra variabile e indirizzo data da:

$$b \rightarrow \$s3$$

la traduzione Assembly è data da:

```
add  $s3,  $s3,  $s3        #  calcola  b = b + b
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'esecuzione dell'istruzione determina la cancellazione dell'operando per effetto della scrittura del risultato nel registro che lo contiene.

Ad esempio, supponendo che il valore dell'operando da sommare a se stesso sia:

$$b = 600 \quad \Longrightarrow \quad b = b+b = 600 + 600 = 1200$$

si ha il seguente contenuto nei registri prima e dopo l'esecuzione dell'istruzione:

INDIRIZZI	BLOCCO DEI REGISTRI PRIMA	BLOCCO DEI REGISTRI DOPO
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*
$s3 ⟷ 19	600	1200
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*

Per semplicità, i registri del processore che non sono coinvolti nell'esecuzione dell'istruzione non vengono indicati nelle figure mostrate perché il contenuto di tali registri rimane invariato dopo l'esecuzione dell'istruzione. A titolo di esempio si mostra il contenuto del blocco completo dei registri relativo all'istruzione sopra descritta:

INDIRIZZI		BLOCCO DEI REGISTRI PRIMA	BLOCCO DEI REGISTRI DOPO
$zero ⟷	0	00000000.....0000000	00000000.....0000000
	1	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	2	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	.....	.....	.....
	18	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
$s3 ⟷	19	600	1200
	20	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	.....	.....	.....
	31	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Addizione: istruzione addi

La seconda istruzione Assembly per la somma è caratterizzata da un diverso modo di individuare gli operandi.

ISTRUZIONE addi

Effettua l'addizione di due operandi interi con segno e memorizza il risultato in un registro; il valore di un operando è contenuto in un registro, il valore dell'altro operando è un numero intero con segno indicato all'interno dell'istruzione.

L'istruzione "addi" corrisponde ad una istruzione del Linguaggio C del tipo:

h = i + 350 ;

Assegnando alla variabile "i" del primo operando ed alla variabile "h" del risultato i registri con i seguenti INDIRIZZI:

h	i	*variabile*
	$t1 ↓	$s0 ↓ *indirizzo simbolico*
	9	16 *indirizzo numerico*

l'istruzione in linguaggio C viene tradotta dalla seguente **istruzione Assembly**:

addi $t1, $s0, 350

Vincoli sintattici:

- Codice Operativo **addi**,
- **l'indirizzo del registro DESTINAZIONE** che conterrà il risultato è il primo, ed è separato da una virgola dal successivo indirizzo,
- **l'indirizzo dell'OPERANDO CONTENUTO NEL REGISTRO** segue l'indirizzo destinazione ed è separato da una virgola dalla successiva costante,
- il **valore dell'OPERANDO COSTANTE** è l'ultima informazione rappresentata e può essere scritto con la **usuale notazione decimale**.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

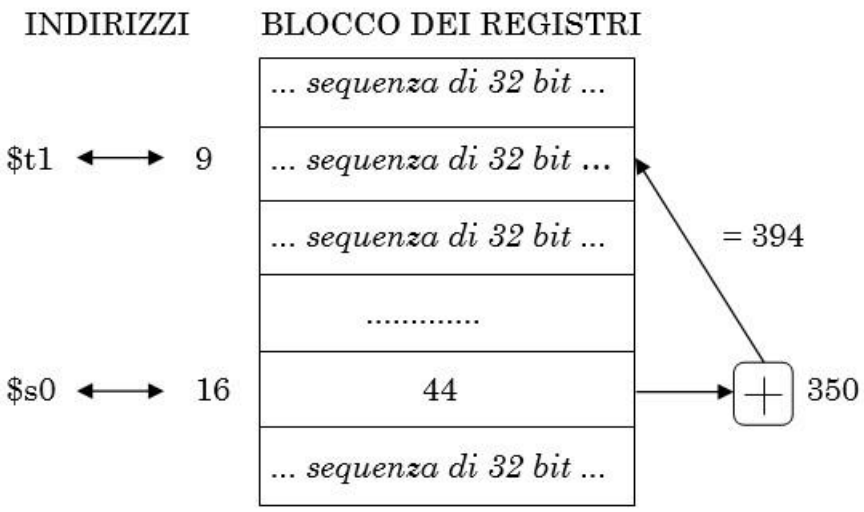
Appositi collegamenti Hardware consentono alla macchina di prelevare il valore del secondo operando dalla relativa parte di bit che lo rappresenta nell'istruzione.

L'esecuzione dell'istruzione ha effetto sul contenuto del registro destinazione, ma lascia immutato il contenuto del registro dell'operando.

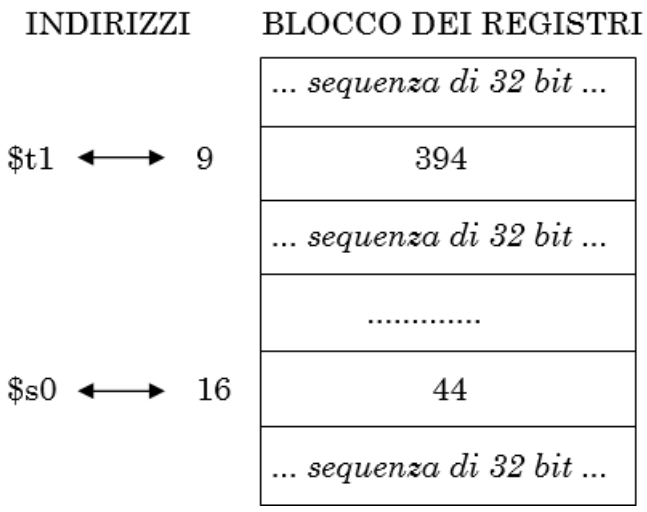
Ad esempio supponendo che il valore dell'operando della precedente istruzione sia:

$$i = 44 \quad \Longrightarrow \quad h = i + 350 = 44 + 350 = 394$$

il contenuto nei registri **PRIMA dell'esecuzione dell'istruzione** è il seguente:



il contenuto nei registri **DOPO l'esecuzione dell'istruzione** è il seguente:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Così come accade nel Linguaggio C, anche nell'Assembly il registro destinazione può essere uguale a quello del primo operando.

ESEMPIO: consideriamo l'istruzione C

$$n = n - 1 \quad ;$$

con l'assegnazione tra variabile e indirizzo data da:

$$n \longrightarrow \$t0$$

la traduzione Assembly è data da:

```
addi  $t0,  $t0,  –1        # calcola  n = n + (–1)
```

L'esecuzione dell'istruzione determina la cancellazione di uno degli operandi per effetto della scrittura del risultato nel registro che lo contiene.

Ad esempio, supponendo che il valore del primo operando sia:

$$n = 500 \quad \Longrightarrow \quad n = n + (-1) = 500 + (-1) = 499$$

si ha il seguente contenuto nei registri prima e dopo l'esecuzione dell'istruzione:

INDIRIZZI	BLOCCO DEI REGISTRI PRIMA	BLOCCO DEI REGISTRI DOPO
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*
$t0 ⟷ 8	500	499
	... sequenza di 32 bit ...	*... sequenza di 32 bit ...*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Sottrazione: istruzione sub

Tutto quello che si è detto per l'istruzione con Codice Operativo "add" relativa all'addizione si applica in maniera analoga all'istruzione dell'Assembly MIPS con Codice Operativo "sub" relativa alla sottrazione.

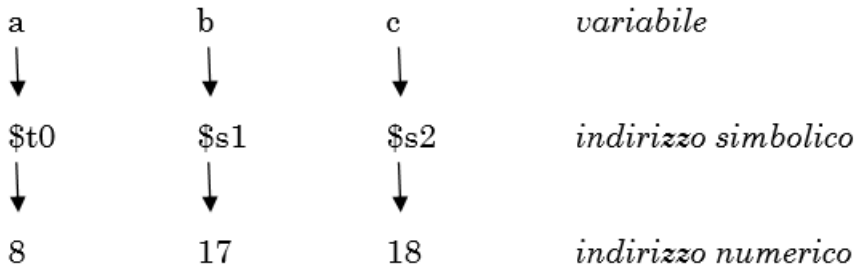
ISTRUZIONE sub

Effettua la sottrazione di due operandi interi con segno il cui valore è memorizzato in due registri e scrive il risultato in un registro.

La sottrazione è una operazione disponibile come istruzione in tutti i Linguaggi ad ad Alto Livello. Nel **Linguaggio C** è codificata da una istruzione del tipo:

$$a = b - c \quad ;$$

Assegnando alle variabili "a", "b", "c" i registri con i seguenti INDIRIZZI:



l'istruzione in Linguaggio C viene tradotta dalla seguente **istruzione Assembly**:

```
sub  $t0, $s1, $s2
```

Vincoli sintattici:

- Codice Operativo **sub,**
- l'**indirizzo del registro DESTINAZIONE** che conterrà il risultato è il primo, ed è separato da una virgola dal Codice Operativo,
- gli **indirizzi dei due REGISTRI DEGLI OPERANDI** seguono l'indirizzo destinazione anch'essi separati da una virgola,
- **ulteriore vincolo sull'ORDINE DEGLI OPERANDI**:

l'indirizzo dell'operando ***minuendo*** PRECEDE quello dell'operando ***sottraendo***.

Nell'esempio precedente la variabile "b" è il *minuendo* e il relativo indirizzo precede quello della variabile "c" che è il *sottraendo*.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Poiché la filosofia di progetto tende a semplificare l'insieme delle istruzioni, l'Assembly MIPS non prevede un'istruzione per la sottrazione di un operando costante in quanto può essere ottenuta con l'istruzione "addi" sommando l'opposto della costante.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Esercizi

ESERCIZIO 1

La scrittura del valore di una costante in un registro è equivalente alla traduzione in Assembly di un'istruzione in Linguaggio C del tipo:

a = –75 ;

Poiché il numero zero è l'elemento neutro dell'addizione, la somma di 0 con una costante è uguale alla costante stessa.

Il registro di indirizzo 0 (indicato con $zero) è riservato a contenere il valore 0, quindi, per scrivere una costante in un registro lo si può porre come registro destinazione nell'istruzione "addi", indicando come operandi il registro $zero e la costante da scrivere.

Assegnando alla variabile il registro con il seguente indirizzo:

a → $s0

l'istruzione viene tradotta dalla seguente **istruzione Assembly**:

```
addi  $s0, $zero, –75     # nel registro destinazione $s0
                          # viene scritta la costante –75
```

e si ha il seguente contenuto nel blocco completo dei registri:

INDIRIZZI		BLOCCO DEI REGISTRI **PRIMA**	BLOCCO DEI REGISTRI **DOPO**
$zero ⟷	0	00000000.....0000000	00000000.....0000000
	1	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	2	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	.....	.....	.....
	15	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
$s0 ⟷	16	*... sequenza di 32 bit ...*	–75
	17	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*
	.....	.....	.....
	31	*... sequenza di 32 bit ...*	*... sequenza di 32 bit ...*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESERCIZIO 2

Caso particolare dell'ESERCIZIO 1 è l'azzeramento di un registro, che equivale alla traduzione in Assembly MIPS dell'istruzione in Linguaggio C:

k = 0 ;

Come in molte situazioni, la traduzione di questa istruzione può essere ottenuta in diversi modi descritti negli esempi seguenti.

ESEMPIO A uso della istruzione "addi" visto nell'ESERCIZIO 1:

```
addi  $s2,  $zero,  0        # addizione tra il contenuto del registro $zero
                             # e la costante 0 indicata nell'istruzione.
```

ESEMPIO B uso della istruzione "add":

```
add  $s2,  $zero,  $zero     # addizione del contenuto del registro $zero
                             # con se stesso.
```

ESEMPIO C uso della istruzione "sub":

```
sub  $s2,  $zero,  $zero     # sottrazione del contenuto del registro $zero
                             # con se stesso.
```

ESERCIZIO 3

Il cambiamento di segno del valore memorizzato in un registro equivale alla traduzione in Assembly MIPS dell'istruzione in Linguaggio C:

b = –b ;

Assegnando alla variabile il registro con il seguente indirizzo:

b ⟶ $t5

può aversi la seguente traduzione in Assembly MIPS:

```
sub  $t5,  $zero,  $t5       # per cambiare il segno del valore
                             # contenuto nel registro $t5 si effettua il calcolo:
                             #  – contenuto in $t5 = contenuto in $zero –
                             (contenuto in $t5).
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).